Predicting Bitcoin's Hourly Price Changes: The Direction Is Noise, the Size Is Not

Alberto Prats
MSc AI, IAI600
University West, Trollhättan, Sweden

Adrián Perera Moreno
MSc AI, IAI600
University West, Trollhättan, Sweden

Filip Botlo
MSc AI, IAI600
University West, Trollhättan, Sweden

Adrián Muñiz Dos Santos
MSc AI, IAI600
University West, Trollhättan, Sweden

***Abstract*—We study how to forecast the 24 hourly price changes of Bitcoin's next day from the preceding 30 days of hourly data. On 59,331 consecutive hourly Bitstamp candles (January 2020 – October 2026), we build scale-free features, split the data into chronological train, validation and test blocks with a purge between them, make every choice without the test block, and compare regularised linear models, nearest neighbours, support vector machines, tree ensembles, neural networks (MLP, LSTM) and a pretrained time-series foundation model (Timestral, zero-shot) against simple rules, with block-bootstrap confidence intervals. No model forecasts the hourly returns better than "no change": at the hourly scale Bitcoin behaves like a random walk, and only the next hour carries a small reversal signal (AUC 0.538). The *size* of the moves, however, is clearly predictable. Gradient Boosting refitted monthly explains 14.8% of the variance of each hourly absolute return (2.6 times a volatility rule), an LSTM fine-tuned monthly 15.6%, zero-shot Timestral 15.7%, and the average of Timestral and Gradient Boosting 16.4%. Conformally calibrated 80% ranges cover 80% of the test hours, and next-day volatility is predicted with $R^2 \approx 0.51$. Keeping models up to date mattered more than any extra feature or data source.**

***Index Terms*—Bitcoin, time-series forecasting, volatility, gradient boosting, conformal prediction, LSTM, foundation models**

I. Introduction

The assigned task is to predict, for each of the 24 hours of the next day, how much the price of Bitcoin will change, $r_h = \mathrm{Close}_h / \mathrm{Close}_{h-1} - 1$, from the hours before. We forecast changes rather than prices because the price level is not stationary: it rose from about $4,000 (March 2020) to over $126,000, and 72% of the test hours trade above the highest training price, so a model of the level would have to extrapolate.

Short-horizon forecasts matter for risk management, position sizing and the timing of trades, and Bitcoin is a demanding case: it trades 24/7, its hourly returns are heavy-tailed, and its behaviour changes between bull and bear markets and after events such as the launch of the US spot ETFs in January 2024. Two classical results frame the problem. In an efficient market, past prices should not predict the direction of future returns [1]; the size of the moves, on the other hand, clusters in time and is predictable [2], [3]. Financial time series also make it easy to fool oneself: overlapping windows and random splits leak the future into the training data [4].

Our contributions are: (i) a leakage-free evaluation in which every choice is made without the test block and every model is compared with a simple rule; (ii) evidence that hourly returns are not predictable beyond a tiny one-hour reversal, while their size is; (iii) calibrated 80% ranges for every hour of the next day; and (iv) a comparison of classical models, neural networks and a zero-shot foundation model on identical rows. Section II describes the data, analysis and models, Section III the setup, Section IV the results and Section V concludes.

II. Methodology

A. Dataset and Source

The data is the Kaggle "Bitcoin Historical Data" set [5] of 1-minute Bitstamp BTC/USD candles, pinned to one version so that every run sees the same file, and aggregated to hourly candles (`Open`, `High`, `Low`, `Close`, `Volume`). The models use the 59,331 consecutive hours from 1 January 2020 to 8 October 2026. Auxiliary sources were tested as extra inputs: hourly BTC and ETH candles with order flow from Binance [6], about 40 daily to monthly macro and market series from FRED [7] and Yahoo Finance [8], and 281 US release times (CPI, payrolls, FOMC) from the BLS and Federal Reserve calendars.

B. Preprocessing

Minutes are aggregated to hours without dropping missing rows, so the hourly grid stays continuous. The minute data hides gaps: a minute without trades appears as a flat, zero-volume candle at the previous close. Such minutes make up 95% of 2012 and 25–40% of 2013–2016, against 1–4% per year since 2018, so earlier years were discarded (an early experiment confirmed that training from 2012 hurts the volatility models). We start in 2020 so that the training block is closer to the current market regime. The remaining outage hours are flagged, not removed; Bitstamp and Binance hourly returns correlate at 0.993, so the data is clean. Scaling is done inside each model pipeline, fitted on training data only.

C. Exploratory Data Analysis

Analyses that could steer a model use the training block only. Hourly returns are heavy-tailed (excess kurtosis 55; 0.41% of the hours lie beyond five standard deviations). Fig. 1 shows the central fact of the project: the autocorrelation of returns is tiny (about -0.03 at lags 1, 2 and 24; an AR(24) model has validation $R^2 = -0.002$), while the absolute returns are strongly autocorrelated (0.30 at lag 1, still 0.10 a week later; an AR(24) on $|r|$ explains 10% of the validation variance). The direction is almost noise; the size is not.

The size also follows the clock, and this rhythm drifts (Fig. 2): the 14 UTC peak (US session) grew from 1.17 times the average hour in 2020 to 1.70 in 2024–2025, and weekend hours went from 0.8 to about 0.6 times weekday hours. Volatility regimes persist: the next day moves 0.32% per hour after calm weeks and 0.71% after wild ones, and a K-Means clustering [9] of the days finds a crash cluster after which the next day moves 1.02% per hour (0.42% after quiet days). Regimes tell how much Bitcoin will move, not which way. ETH moves with Bitcoin in the same hour (correlation 0.84) but does not lead it.

The macro variables were compared with Spearman correlations and a false-discovery-rate correction [10]. Bitcoin follows global risk appetite (S&P 500: 0.33 daily, 0.53 monthly; VIX -0.29 daily), but in lead–lag tests using only values public at the time, no variable leads it, and the links change from year to year. US releases do move Bitcoin: the release hour moves 2.2–2.5 times the same hour on nearby days (FOMC up to 3.6 times; Wilcoxon $p < 0.001$ on train and validation). From these findings, 18 candidate features (volatility ratios, release-hour flags, a 90-day hour profile, VIX) were built, checked for look-ahead and screened on train and validation.

D. Prediction Targets and Features

A forecast is made at every hour (the *origin*) with 30 days of history before it and 24 target hours after it. Besides the assigned target, the 24 hourly returns, we ask the questions the data can answer: (a) the direction of the next hour; (b) the size $|r|$ of each of the next 24 hours; (c) an 80% prediction range for each of these hours; and (d) the next day's return, direction and volatility. All features are scale-free, so they mean the same in 2020 and 2026: lagged returns, realised volatilities over 24, 72 and 168 hours, high–low ranges, volume ratios and the calendar. In the *long format* each origin gives 24 rows, one per predicted hour, with per-hour columns (the average size of that hour, its size one and seven days earlier).

E. Models

Classical: Ridge [11], Lasso [12], ElasticNet [13], k-nearest neighbours, SVR/SVC, Random Forest [14], Extra Trees [15], Gradient Boosting (GB) [16] and averages of them. Ranges come from GB with the quantile loss [17] for the 10% and 90% quantiles, recalibrated by *rolling conformal calibration* [18], [19]: the half-width is multiplied by the factor that would have given 80% coverage over the last 30 days of known outcomes; a *Mondrian* variant calibrates release hours apart. For the volatility targets, GB is also refitted every month on all past data, with weights that halve every 12 months. *Neural:* MLPs (scikit-learn, PyTorch, Keras) and LSTMs [20] reading a 168-hour window of ten window-normalised channels, trained with MSE or Huber loss. The best LSTM also receives the clock and release flags of each predicted hour, weights the week through an attention mask built from traded volume, and is fine-tuned every month. *Foundation model:* Timestral 28M [21], a pretrained time-series transformer that returns 21 quantiles of each of the next 24 hours. It is used zero-shot: its weights never see our data; we only choose, on validation, the series it reads (r or $|r|$), the context length (30–180 days), the covariates and the model size. We also fitted GARCH(1,1) [3] as a classical volatility benchmark.

F. Rationale for the Chosen Methods

The zero forecast and simple rules (the reversal of the last hour, last week's volatility times the hour profile) set the bar: a model is only useful if it beats them. Regularised linear models suit a target that is mostly noise; tree ensembles capture the non-linear effects of volatility and the clock; networks test whether a sequence model finds what the hand-made features miss; and the foundation model tests whether pretraining on many series can replace fitting on ours. Monthly refits and conformal calibration answer the drift seen in the analysis. No large language model is part of the forecasting solution.

III. Experimental Setup

A. Tools and Environment

All code is Python 3.12 with NumPy [22], pandas [23], SciPy, scikit-learn [24], PyTorch [25], TensorFlow/Keras [26], the `timestral` package [21] and Matplotlib [27]; versions are pinned in `requirements.txt`. The work consists of five Jupyter notebooks (base data, data analysis, classical models, neural networks, Timestral) sharing a `utils` package for the split dates, features, metrics and statistical tests. The classical notebook runs in about 45 minutes on 16 CPU cores; the neural notebook in 33 minutes and the Timestral notebook in under two hours on a laptop GPU (RTX 5060). Code: https://github.com/Descalabreitor/IAI600-final-project.

B. Data Split

The data is split into three chronological blocks by the date of the predicted hours (Table I). A 192-hour purge before each border (the 168-hour input window plus the 24 target hours) ensures that no input or target hour of one block falls into the next [4]. Train and validation use every sixth origin; the test block uses every origin, and daily statistics use one forecast per day at 23:00 (829 independent test days).

C. Model Selection and Hyperparameter Tuning

Every choice (model, hyperparameters, features, refit schedule, calibration, Timestral setting) is made on train and validation only; the chosen model is refitted on both and scored once

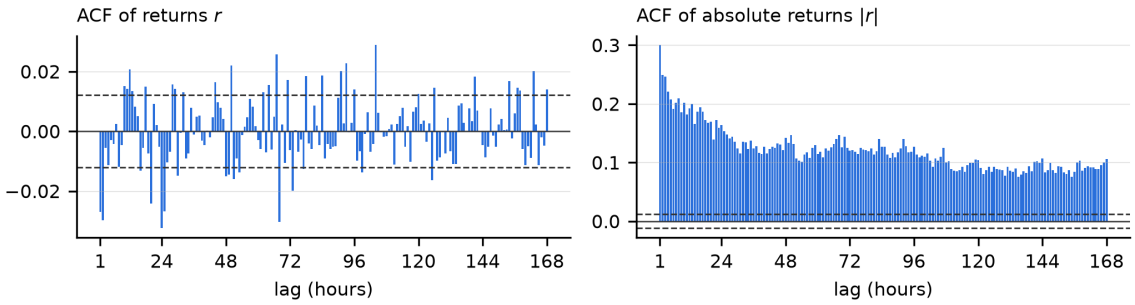


Fig. 1. Autocorrelation of hourly returns (left) and of their absolute value (right), lags 1–168 h, training block 2020–2022. Dashed lines: 95% band of white noise.

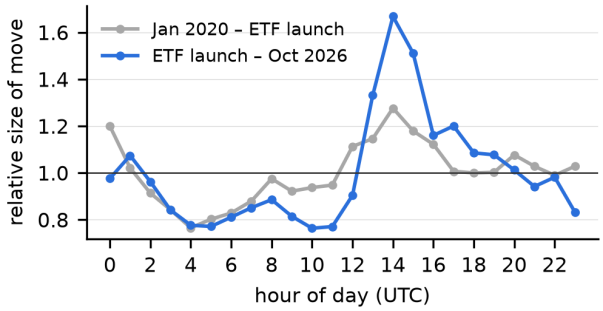


Fig. 2. Size of the hourly move by hour of day (UTC), relative to the average hour, before and after the launch of the US spot ETFs (11 Jan 2024).

TABLE I
CHRONOLOGICAL DATA BLOCKS (192-HOUR PURGE BEFORE EACH BORDER).

Block	Predicted hours	Hours	Use
Train	Jan 2020 – Dec 2022	26,304	fitting
Validation	Jan 2023 – Jun 2024	13,128	choice of model and settings
Test	Jul 2024 – Oct 2026	19,899	scored once

on test. Hourly targets are tuned on the validation block, daily targets with four walk-forward folds over train and validation, which show the true spread between periods (± 0.08 R^2). Grids are small and extended when the best value lies on an edge; a random search gave the same result as the full grid at a quarter of the cost. The number of boosting trees is tuned directly: early stopping on a random hold-out used up to 2.4 times too many. Models are selected by the reported metric, which matters for returns: a model trained for the mean but chosen by MAE is pushed towards the zero forecast. Networks stop early on the last six months of train, never on a random hold-out, and are averaged over five seeds.

D. Evaluation Metrics and Baselines

Returns are scored by the MAE skill against the zero forecast ($1 - \mathrm{MAE}/\mathrm{MAE}_0$), by R^2_{oos} (the R^2 against the zero forecast) and by the information coefficient (IC, the correlation of forecast and outcome). Direction is scored by the AUC and by the accuracy of the 10% most confident calls (binomial test); size and volatility by R^2; ranges by coverage, width, pinball loss and CRPS [28]. Baselines are the zero forecast, the reversal of the last hour, last week's volatility times the hour profile, and "always up". Uncertainty is measured by a moving-block bootstrap [29] over the test days (blocks of 10 days, 1,000 resamples) and Diebold–Mariano tests [30]; a difference is called *reliable* when its 95% interval excludes zero.

IV. Results and Discussion

Table II summarises the test results; every model in it was chosen without the test block.

A. Hourly Returns: the Assigned Task

No model forecasts the 24 hourly returns better than "no change". Every tuned model ends within 0.0001 percentage points of the MAE of the zero forecast (0.3193%), the long-format models have R^2_{oos} within ± 0.001 of zero (Diebold–Mariano $p \geq 0.15$), and the networks and the median of Timestral do no better. More model freedom always meant worse validation scores, the signature of a target that is mostly noise. Selecting by the matching metric raises the IC from 0.003 to 0.021, and its interval $[+0.006, +0.035]$ excludes zero, but the ranking signal is too weak to make the forecast itself beat zero. Direction accuracy is easily misread: on this balanced test block (50.1% up days) every model scores 49.9–50.1%. At the hourly scale Bitcoin is close to a random walk, as Fig. 1 suggested.

B. Next-Hour Direction

The one exception is the next hour. The rule "the next hour moves against the last one" reaches AUC 0.528 and is right 54.6% of the time on its 10% most confident calls ($p = 2 \cdot 10^{-5}$); GB with Binance order-flow features adds a little (AUC 0.538, 56.0%). An LSTM reaches the same AUC only when trained on every origin, which helps GB as much (0.541), so the data, not the network, makes the difference. The effect is real but small, measured before trading costs, and vanishes from the second hour on: hours 2–24 and the next day are a coin flip.

TABLE II
TEST-BLOCK RESULTS (JUL 2024 – OCT 2026) OF THE MODELS CHOSEN ON TRAIN AND VALIDATION, AGAINST THE BEST SIMPLE RULE. BRACKETS: 95% BLOCK-BOOTSTRAP INTERVALS.

Question	Chosen model	Test result	Simple rule	Beats rule?
24 hourly returns (assigned task)	tuned linear, KNN, trees, MLP, LSTM	MAE skill -0.0004 to -0.0002 (networks to -0.0020); IC ≤ 0.021; DM $p \geq 0.15$	zero forecast	no
Direction of the next hour	GB classifier (+ Binance); LSTM	AUC 0.538; 56.0% right on top-10% calls ($p = 5 \cdot 10^{-8}$)	last-hour reversal: AUC 0.528	yes, small
Size $\lvert r \rvert$ of each next hour	Timestral + GB refitted monthly	$R^2 = \mathbf{0.164}$ (Timestral 0.157, LSTM 0.156, GB 0.148)	last week $\times$ hour profile: 0.056	yes
80% range for each next hour	GB quantile + rolling conformal; Timestral	coverage 80.0% (calm 78%, wild 82%), width 0.98%; Timestral 80.3%, 1.00%, uncalibrated	fixed range: 87.6%, 1.28%	yes
Next-day return	Ridge	$R^2_{\text{oos}} = -0.001$, IC 0.051	zero forecast	no
Next-day direction	Random Forest	AUC 0.509, accuracy 49.9%	always up: 50.1%	no
Next-day volatility	top-3 tree ensemble; GB monthly	$R^2 = 0.510$ [0.45, 0.56]; refitted monthly 0.517	last week's mean: 0.094	yes
Volatility targeting (economic)	next-day volatility forecast	Sharpe 0.27, max. drawdown 64%	buy & hold: 0.50, 53%	no

C. Size of the Hourly Moves

The size of the moves is where the models have clear skill (Fig. 3). Moving from one row per origin to the long format was the largest modelling gain (validation R^2 0.052 to 0.097). On test, GB fitted once explains 13.1% of the variance of each hourly $|r|$, against 5.6% for the volatility rule (gain $[+0.057, +0.084]$); its skill falls from about 0.16 one hour ahead to 0.11 at 24 hours. Refitting every month with a 12-month half-life raises R^2 to 0.148 ($+0.017$ $[+0.010, +0.024]$, better in 20 of 29 months): with a drifting market (Fig. 2), keeping the model current is worth more than any extra input. None of the candidate features from the analysis (volatility ratios, release hours, recent hour profile), nor Binance and ETH, improved the trees (-0.0004 $[-0.004, +0.003]$ after refitting). Release hours stay under-predicted (0.65 times their actual size): with 544 training rows they are fewer than one leaf of the tuned GB, and a correction factor learnt from past errors over-shot on test.

Networks gain only with care. An LSTM on the raw window reaches 0.100; telling it the clock and release flags of each predicted hour adds $+0.031$, the volume attention mask $+0.014$ and monthly fine-tuning $+0.011$, all reliable, giving 0.156, narrowly above the refitted GB ($+0.008$ $[+0.001, +0.015]$). The MLP only ties the trees.

Timestral, used zero-shot, matches this without being trained on our data: 0.157 ($+0.009$ $[+0.002, +0.016]$ over refitted GB). What it reads matters more than its size: the series $|r|$ rather than r (validation 0.108 vs. 0.067), a long context (0.088, 0.105 and 0.108 for 30, 90 and 180 days) and the release calendar as known-future input ($+0.008$); the clock adds nothing because the model finds the hour profile in its context. The 209M model added only 0.003 for seven times the compute. ETH, VIX, order flow and a GARCH forecast as covariates changed validation by less than 0.001 and nothing on test. The plain average of Timestral and refitted GB, chosen on validation, is the best result of the project: $R^2 = 0.164$ ($+0.016$ $[+0.013, +0.020]$). Stacking Timestral's forecast as a GB feature was worse (0.155–0.157): the trees map it with what they learnt on the wilder 2020–2022 hours. GARCH(1,1) alone is weaker than the simple rule (validation 0.005 vs. 0.050), because its likelihood is driven by a few huge hours rather than the typical move.

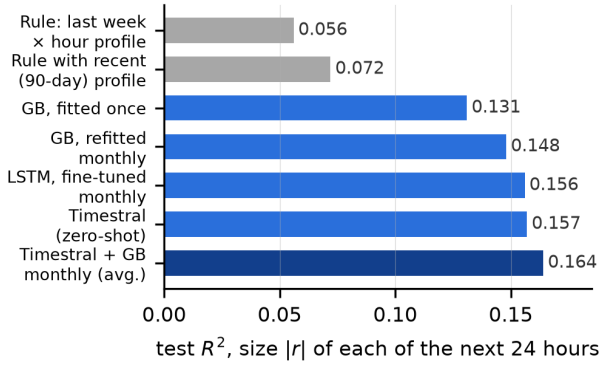


Fig. 3. Test R^2 for the size of each of the next 24 hourly moves: simple rules (grey), single models (blue) and the chosen average of Timestral and monthly-refitted Gradient Boosting (dark blue).

D. Prediction Ranges

The GB quantile ranges are 28% narrower than a fixed range (0.92% vs. 1.28%) and adapt to the market, but cover only 77.8% of the test hours: the test years differ from the training years. A correction measured once on validation does not fix this (77.3%), whereas rolling conformal calibration does: 80.0% coverage, 78% in calm and 82% in wild hours, at a width of 0.98%, while the fixed range over-covers calm hours (95%) and under-covers wild ones (80%). Release hours are the weak spot (56% coverage); calibrating them apart (Mondrian) lifts them to 82% with ranges 2.08% wide there. Timestral's own quantiles need no calibration: 80.3% coverage at 1.00% width, even across calm and wild hours, every one of its 21 quantile levels within 1.4 points of its nominal level, the lowest CRPS of the ranges tried (0.236 vs. 0.240 for a volatility-scaled baseline), and 70% coverage in release

hours ($+13.8$ points $[+7.9, +18.9]$ over GB with plain rolling conformal).

E. Next-Day Targets and Economic Test

At the daily scale, return and direction are not predictable (AUC interval $[-0.023, +0.046]$ around 0.5), but volatility is: a top-3 tree ensemble explains 51% of the next day's log volatility, against 9.4% for last week's average, and a monthly refit adds $+0.015$ $[+0.004, +0.025]$. On a small daily table, plain Linear Regression ($R^2 = 0.514$) and SVR (0.519) beat Random Forest (0.464) and a Keras MLP (0.399), because tomorrow's volatility depends almost linearly on today's. The average of GB and an MLP beats GB alone ($+0.011$ $[+0.001, +0.022]$); the MLP alone ties. Learning curves agree with the refit results: volatility models do better with only the last six months of data, return models need the whole history. A good forecast is not a good strategy, though: volatility targeting with this forecast (leverage capped at 2) reaches a Sharpe ratio of 0.27 after costs against 0.50 for buy-and-hold, and already lags before costs (0.41), because of its design (target level set on the more volatile training years, leverage, turnover).

F. Discussion

The results split the task in two. The sign of the hourly changes is unpredictable except for a small reversal one hour ahead; their size, and therefore the range in which the price will move, is predictable and can be stated with honest coverage. Because Bitcoin's behaviour drifts, models and calibrations that are kept current beat models with more data or more inputs: a feature that carries information on its own (release hours move three times more) does not improve a model that already sees recent volatility and the clock, or that has too few examples to use it. Model families converge: GB, a carefully built LSTM and an untrained foundation model end within 0.01 R^2, and averaging two different families gives the best result. Two caveats apply: there is a single test period of 2.25 years, and Timestral was released after our test block without a list of its training series; if Bitcoin 2023–2026 is among them, its scores are optimistic.

V. Conclusion

The 24 hourly price changes of Bitcoin's next day cannot be forecast better than "no change": at the hourly scale the price is close to a random walk, and only the next hour carries a small reversal signal (AUC 0.54). The size of the changes is predictable, so we recommend answering the task with the expected size and an 80% range for each hour. The average of zero-shot Timestral and monthly-refitted Gradient Boosting explains 16.4% of the hourly variance (2.9 times a volatility rule); conformally calibrated ranges cover 80% of the hours in calm and wild markets alike; and next-day volatility is predicted with $R^2 \approx 0.51$.

We learned that the evaluation design matters more than the choice of model: chronological blocks with a purge, choices made without the test block, comparison with simple rules and bootstrap intervals are what separate a real signal from noise. Recency matters more than data volume or extra sources: monthly refits were the largest single gain, while Binance, ETH, macro variables and new features added nothing reliable. A pretrained model, used as it comes, can match careful feature engineering where there is signal, but cannot create signal where there is none.

The study has limitations: a single test period; many tests on the return target without a multiple-testing correction; no trading costs in the next-hour signal; release hours that remain hard to forecast; and a possible overlap between Timestral's training data and our test period. Future work includes a rolling-origin evaluation over 2023–2026, Holm or Benjamini–Hochberg corrections, a cost-aware test of the next-hour signal, a forward test of Timestral on data after its release, monthly fine-tuning of Timestral, and a volatility-targeting design tuned on validation.

Acknowledgment

Use of generative AI. Claude Code (Anthropic) was used for assistance in coding, debugging, refactoring, git operations, wording of notebook texts and drafting of this report. No language model is part of the forecasting solution: Timestral is a time-series model, used zero-shot.

Contributions. Alberto Prats: classical models and hyperparameter tuning; Adrián Perera: data pipeline, unification of the notebooks, neural networks and Timestral; Filip Botlo: Binance features, monthly refits, conformal ranges, course-method pipeline; Adrián Muñiz: exploratory data analysis and macro correlations. We thank the author of the Kaggle dataset, Binance, FRED and Mistral AI for the public data and models, and the course staff.

References

[1] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *J. Finance*, vol. 25, no. 2, pp. 383–417, 1970.

[2] R. F. Engle, "Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation," *Econometrica*, vol. 50, no. 4, pp. 987–1007, 1982.

[3] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *J. Econometrics*, vol. 31, no. 3, pp. 307–327, 1986.

[4] M. López de Prado, *Advances in Financial Machine Learning*. Hoboken, NJ, USA: Wiley, 2018.

[5] Z. Zielinski, "Bitcoin Historical Data," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/mczielinski/bitcoin-historical-data

[6] Binance, "Binance public data." [Online]. Available: https://data.binance.vision

[7] Federal Reserve Bank of St. Louis, "FRED economic data." [Online]. Available: https://fred.stlouisfed.org

[8] R. Aroussi, "yfinance." [Online]. Available: https://github.com/ranaroussi/yfinance

[9] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probab.*, 1967, pp. 281–297.

[10] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate," *J. R. Statist. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.

[11] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[12] R. Tibshirani, "Regression shrinkage and selection via the lasso," *J. R. Statist. Soc. B*, vol. 58, no. 1, pp. 267–288, 1996.

[13] H. Zou and T. Hastie, "Regularization and variable selection via the elastic net," *J. R. Statist. Soc. B*, vol. 67, no. 2, pp. 301–320, 2005.

[14] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, 2001.
[15] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Mach. Learn.*, vol. 63, pp. 3–42, 2006.
[16] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
[17] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.
[18] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.
[19] I. Gibbs and E. Candès, "Adaptive conformal inference under distribution shift," in *Proc. NeurIPS*, 2021, pp. 1660–1672.
[20] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
[21] Mistral AI, "Timestral: time-series foundation models," 2026. [Online]. Available: https://github.com/mistralai/timestral
[22] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
[23] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, 2010, pp. 56–61.
[24] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[25] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019, pp. 8024–8035.
[26] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX OSDI*, 2016, pp. 265–283.
[27] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.
[28] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *J. Amer. Statist. Assoc.*, vol. 102, no. 477, pp. 359–378, 2007.
[29] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *Ann. Statist.*, vol. 17, no. 3, pp. 1217–1241, 1989.
[30] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," *J. Bus. Econ. Statist.*, vol. 13, no. 3, pp. 253–263, 1995.